Collaborative Tag-Aware Graph Neural Network for Long-Tail Service Recommendation

Zhipeng Zhang, Yuhang Zhang, Mianxiong Dong, *Member, IEEE*, Kaoru Ota, *Member, IEEE*, Yao Zhang, and Yonggong Ren

***Abstract*—Long-tail service recommendation provides an unexpected but reasonable experience for potential developers when they construct mashups. However, the lack of available information makes it difficult to recommend highly relevant long-tail services for target mashups. Collaborative tagging systems employ extensive tag records to replenish the available information of long-tail services, whereas existing tag-aware approaches are unable to learn multi-aspect embeddings from graphs with different structures and relationships for long-tail services. To this end, we present a novel approach, namely collaborative tag-aware graph neural network, to recommend satisfactory long-tail services by extracting multi-aspect embeddings. First, a tensor decomposition is executed to parameterize mashups, tags, and services as low-dimensional vector representations, respectively. Then, an interaction-aware heterogeneous neighbor aggregation is presented to aggregate both neighboring node features and interaction strength to enhance the embedding quality of long-tail services. Next, a diffusion-aware homogeneous neighbor aggregation is proposed to assign higher weights for long-tail neighboring nodes so as to reduce the influence of popular neighboring nodes during the aggregation process. Furthermore, a type-aware attention network is employed to update the final node embedding by aggregating multi-aspect embeddings. Experimental results on two real-world Web service datasets indicate that the proposed approach generates superior accuracy and diversity than state-of-the-art approaches in the aspect of long-tail service recommendation.**

***Index Terms*—Collaborative tagging, graph neural network, attention mechanism, long-tail service recommendation.**

Manuscript received 10 December 2022; revised 20 July 2023; accepted 25 July 2023. Date of publication 4 January 2024; date of current version 9 October 2024. This work was supported in part by JSPS KAKENHI under Grants JP19K20250, JP20F20080, JP20H04174 and Leading Initiative for Excellent Young Researchers (LEADER), MEXT, Japan, in part by the NSFC under Grant 61976109, in part by the Liaoning Revitalization Talents Program under Grant XLYC2006005, in part by the Scientific Research Project of Liaoning Province under Grant LJKZ0963, in part by the Key R&D projects of Liaoning Provincial Department of Science and Technology under Grant 2022JH2/101300271, in part by Liaoning Province Ministry of Education under Grant LJKQZ20222431, in part by CSC under Grant 202108210173, in part by Industry-University Collaborative Education Project of the Ministry of Education under Grant 202102550005. *(Corresponding authors: Mianxiong Dong and Yonggong Ren.)*

Zhipeng Zhang, Yuhang Zhang, and Yonggong Ren are with the School of Computer Science and Artificial Intelligence, Liaoning Normal University, Dalian 116081, China (e-mail: zhipengzhang@lnnu.edu.cn; zhangyuhang65214@gmail.com; ryg@lnnu.edu.cn).

Mianxiong Dong and Kaoru Ota are with the Department of Information and Electronic Engineering, Muroran Institute of Technology, Muroran 050-8585, Japan (e-mail: mx.dong@csse.muroran-it.ac.jp; ota@csse.muroran-it.ac.jp).

Yao Zhang is with the School of Mechanical Engineering and Automation, Dalian Polytechnic University, Dalian 116034, China (e-mail: zhangyao@dlpu.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3349853

I. Introduction

RECOMMENDER system (RS) has gradually become a significant tool for service developers with the rapid development of service-oriented computing. The long-tail business model breaks the "Pareto Principle" (i.e., 80/20 rule, that means 20% of products often contribute 80% of sales). Although individual sales of long-tail products are very low, the sum of their sales is huge and can even surpass sales of some hot products. In recent years, the long-tail model also attracts more attention in the field of service computing. For instance, in ProgrammableWeb, a popular Web service ecosystem where developers can create mashups that combine multiple web applications or data sources into a single unified tool, more than half of the mashup developers have shown interest in using long-tail Web services compared to the past [1]. Hence, long-tail service recommendation holds great value for both commerce and research purposes.

Long-tail services can be defined as the services that account for 20% of the total number of service invocations [2], the remaining services can be considered as short-head services (also called popular services). Fig. 1 shows an illustration of long-tail services and short-head services. It is clear that long-tail services comprise the majority of total services. Generally, excellent RSs should be able to recommend a variety of services that cover as much search space as possible. However, according to the statistical analysis in [3], the top 200 short-head Web APIs in ProgrammableWeb possess more than 90% of the total number of invocations from mashups, meaning that only a small portion of Web APIs (e.g., short-head Web APIs) is frequently invoked by mashups, while the overwhelming majority of them (e.g., long-tail Web APIs) have little chance to be invoked. Since developers tend to invoke Web APIs to create mashups according to their historical records, long-tail Web APIs that own few usage are hard to appear in the view of developers. Consequently, long-tail Web APIs that really meet the needs of developers' mashups cannot be recommended to them at the appropriate time.

To improve the long-tail service recommendation performance, current approaches focus on employing side information since it can enrich the available information for long-tail services. Thereinto, tag information is often utilized to replenish the feature information of long-tail services, so that the long-tail services can be precisely depicted from multiple angles. Collaborative tagging systems (CTS) have been popularly employed

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

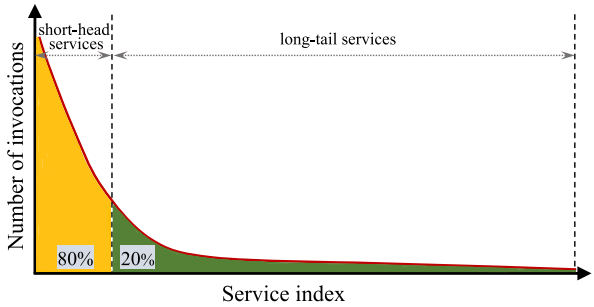


Fig. 1. Illustration of long-tail services and short-head services. Services are sorted by descending order according to the number of invocations.

as a useful tool for managing Web services. Many existing service websites, such as ProgrammableWeb, allow developers to annotate Web APIs and mashups with arbitrary tags. Currently, deep neural networks combined with tagging information are confirmed to be effective in improving the long-tail service recommendation [4], [5], [6], [7]. Specially, graph neural network (GNN) has received widespread attentions, it regards mashups, tags, and services as nodes, and connections between them as edges to construct a collaborative tagging graph (CTG). An example of CTG can be found on the upper left corner of Fig. 2. Then, GNN utilizes graph operators (e.g., graph convolution, graph attention, etc.) on CTG to encode both node features and graph structure [8], [9]. Because GNN can deeply and accurately extract the embeddings of nodes, it has gained remarkable achievements in the long-tail service recommendation [10]. However, existing GNN-related approaches pay more attention to the feature of neighboring nodes, but ignoring their interaction information. In addition, most of related approaches operate GNN on either heterogeneous network or homogeneous network, but not in both, resulting in the lack of a unified framework for the joint use of two types of networks. On the other hand, most existing Web service recommendation approaches often rely heavily on the accurate queries from developers. However, a developer may feel overwhelmed about which keywords are suitable for queries [11].

Motivated by this, we propose a novel collaborative tag-aware graph neural network (CTGNN) which integrates CTG with GNN to extract multi-aspect embeddings for both mashups and long-tail services. Specially, CTGNN is able to recommend satisfying long-tail services for a target mashup according to the mashup query generated by user-selected tags. In order to conduct graph convolution and model training more efficiently, CTGNN first utilizes tensor decomposition to generate low-dimensional embeddings for mashups, tags, and services, respectively. Furthermore, to take full advantage of information contained in both heterogeneous and homogeneous networks, CTGNN utilizes the interaction-aware and diffusion-aware neighbor aggregations to generate three types of embeddings for both mashups and services. Moreover, to discriminate the role of different types of embeddings, a type-aware attention network is executed to assign different weights to different types of embeddings and generate the final embeddings for both mashups and services. After optimizing the recommendation model, top N services that own highest predicted scores and contain as many long-tail services as possible are recommended to target mashups. Fig. 2 indicates the flowchart of CTGNN, and the main contributions of this paper are summarized as follows:

1) A novel CTGNN is presented to merge CTG with GNN for producing satisfying long-tail service recommendation. CTGNN comprehensively aggregates features of neighbors in both heterogeneous and homogeneous networks, and merges different types of features using GNN and attention mechanism to generate multi-aspect embeddings for both mashups and services.
2) An interaction-aware heterogeneous neighbor aggregation is proposed to effectively improve the embedding quality of long-tail services and further enhance the recommendation accuracy by aggregating both neighboring nodes' features and their interactions in a heterogeneous network. Besides, a diffusion-aware homogeneous neighbor aggregation is constructed to improve the recommendation diversity, which executes head diffusion on mashup-tag-service triple network to assign higher weights for long-tail neighbors and reduce the influence of popular neighbors during aggregation process. Experimental results in Section V-E confirm this.
3) A type-aware attention network is presented to update the node embeddings. Different from existing related approaches that directly execute pooling-based operations to update the node information [10], [12], [13], CTGNN assigns different contributions for three types of embeddings by attention mechanism during the final embedding aggregation process.

The rest of this study is arranged as follows. In Section II, we gradually introduce existing related approaches of GNN-based service recommendations, tag-aware recommendations, and long-tail recommendations. Section III describes the main notations used in this paper and gives the problem statement. Section IV systematically introduces the specific contents of our proposed CTGNN, mainly includes mashup-tag-service tensor decomposition, interaction-aware heterogeneous neighbor aggregation, diffusion-aware homogeneous neighbor aggregation, type-aware node embedding updating, and loss function. Section V introduces the experimental results on real-world Web service dataset as well as the comparisons with the state-of-the-art approaches to demonstrate the superior performance of CTGNN on long-tail service recommendation. Finally, Section VI presents conclusions and future work.

II. Related Work

In this section, we give an introduction of the existing related approaches that are determined to employ GNNs to produce satisfactory service recommendation. In addition, we also describe the dominant approaches of tag-aware recommendations and long-tail recommendations.

A. GNN-Based Service Recommendations

GNNs employ the propagation information of neighbors to learn the nodes' representations, and perform aggregation

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

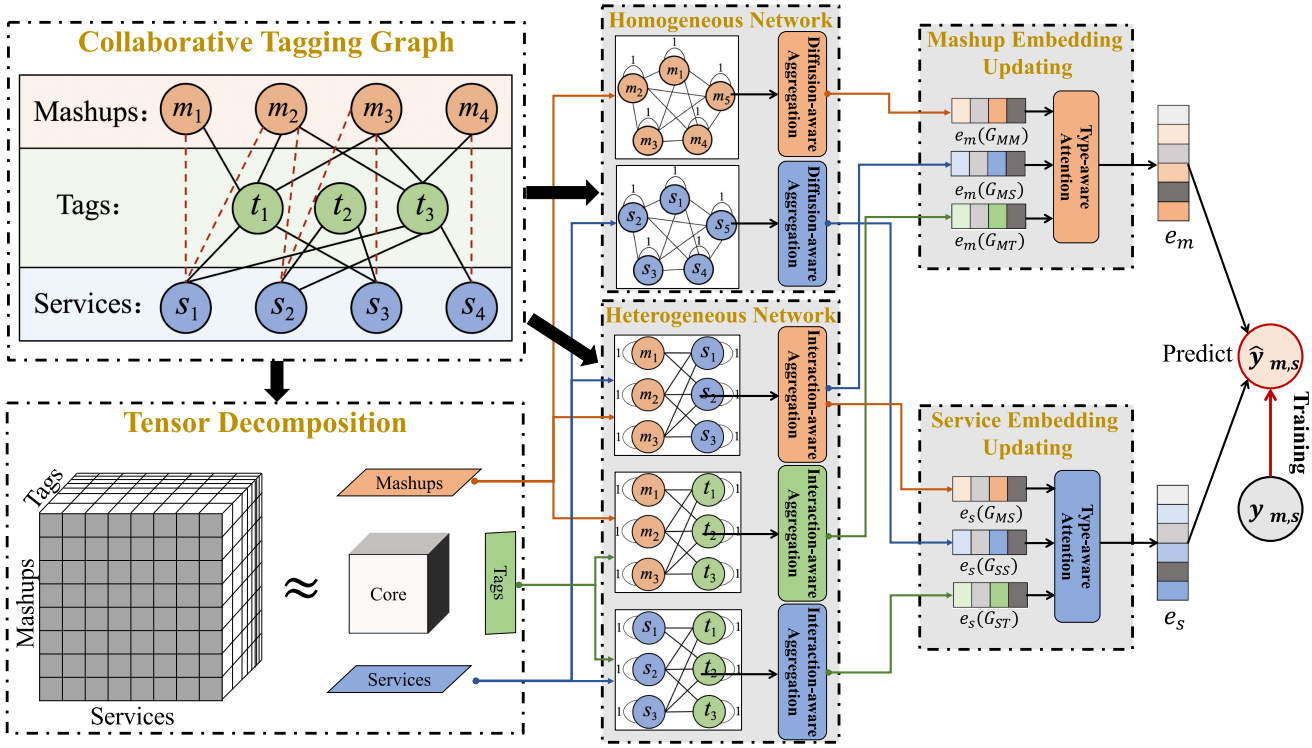


Fig. 2. Flowchart of CTGNN. CTGNN first executes mashup-tag-service tensor decomposition to generate the low-dimensional vector representations of mashups, tags, and services, respectively. Then, it conducts interaction-aware heterogeneous neighbor aggregation on heterogeneous network and diffusion-aware homogeneous neighbor aggregation on homogeneous network respectively, so as to generate three types of embeddings for both mashups and services. Next, it utilizes a type-aware attention network to update the embeddings for both mashups and services. Finally, it merges the final embeddings of mashups and services to predict scores, and utilizes loss function to train the model until obtaining the optimal solution.

operations on the learned embeddings to generate the final embeddings for each target node [8], [9]. Since GNNs can efficiently capture the collaborative information to produce high-quality embedding for each node, they achieve remarkable success in service recommendations. Cao et al. [3] executed service clustering according to the latent topics, and generated satisfying Web service recommendations for mashups through employing the implicit co-invocation relationship. Wei et al. [4] proposed a service-level attentional network to extract users' preferred services from neighbor's mashups, and further employed a friend-level graph attention network to propagate the influence on social network. Mezni et al. [5] utilized the multiple and complex interactions among heterogeneous network to propose a temporal service knowledge graph that can provide satisfying service recommendation. Ren et al. [6] presented a granule distribution-aware support vector machine approach using granular computing to rank unknown services and further generate service recommendations. Fan et al. [7] constructed a context-aware service recommendation model based on the spatial correlations and temporal decay to generate personalized service recommendations. However, existing GNN-based service recommendation approaches suffer from the long-tail service recommendation problems, it is because GNNs generally execute graph convolution on the embeddings of neighbor, thus, the number of neighbors will affect the aggregation quality directly. Whereas the long-tail services are invoked by mashups for only few times. In this case, GNNs cannot generate reliable embeddings for long-tail services through insufficient number of neighbors, making it impossible to produce satisfactory long-tail service recommendations.

B. Tag-Aware Recommendations

CTS permits users to freely annotate on-line services with arbitrary tags, which can not only express users' personalized requirements but also describe the characteristic of services. CTS does not require developers to master specific skills, thus, it can alleviate the limitation of vocabulary and is widely utilized in various of websites and real-world applications. Tag-aware recommender system (TRS) integrates CTS into RS so that rich tag records can be utilized into recommendation process. Recently, TRS with deep learning technology has achieved significant breakthroughs and successes. Chen et al. [12] proposed a tag graph convolutional network by feat of the strong representation ability of GNN, and utilized the feature information of multi-hop neighboring nodes to improve the recommendation performance. Chen et al. [13] utilized a multi-layer perception to map the sparse features into dense latent space, then extracted the representations of users and services using a hierarchical attention network, and fed representations into factorization machines to generate predicted scores. Huang et al. [14] constructed a tag-based attention neural network to extract potential tag embeddings and assign different tag contributions for target users, so as to precisely capture the specific requirements of

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

target users. Pan et al. [15] depicted a user's profile by integrating tag cloud generation and expansion as well as their relationships, so as to construct a sufficient tag set to resolve the tag sparsity issue. Zheng et al. [16] utilized the available semantic tags of music tracks to complement a highly sparse interaction matrix, and presented a tag-aware dynamic recommendation framework that could produce personalized and accurate recommendations to target users. Xu et al. [17] maximized the deep-semantic similarities between users and services, and proposed a deep neural network that mapped the profiles of users and services to a deep feature space so as to accurately track target users' personalized requirements. Shi et al. [18] merged multi-relational network and factorization machines to present a topic-sensitive approach for recommending suitable tags for mashups. However, approaches mentioned above excessively focus on the node feature information contained in CTG, but ignoring the interactions among nodes in CTG. In addition, most of existing approaches tend to assign a same weight when aggregating different types of neighboring nodes, resulting that the characteristics of long-tail services cannot be learned effectively because they are allocated the same weight with other services (e.g., short-head services).

C. Long-Tail Service Recommendations

The interaction data in recommendation systems is observational, rather than experimental, leading to biases in the data [19]. Popularity bias is the most relevant type of bias in long-tail recommendation [20]. Many researchers are currently exploring methods to improve long-tail recommendation performance by addressing popularity bias. Wei et al. [21] proposed a model-agnostic counterfactual reasoning approach to eliminate popularity bias, while Wang et al. [22] presented a new multi-objective training approach for long-tail recommendation through maximizing recommendation diversity. Zhang et al. [23] proposed a causal intervention approach that involves estimating the causal effect of popularity bias. Although these approaches can improve long-tail recommendation performance by eliminating popularity bias, they are only applicable to conventional long-tail recommendations that rely on large amounts of historical interaction data, such as long-tail movies and long-tail products. However, for long-tail service recommendations, which focus on real-time matching between long-tail services and mashup queries, existing approaches for removing popularity bias cannot achieve accurate matching between long-tail services and mashup queries in real-time, and therefore cannot be directly applied to long-tail service recommendations.

Currently, to generate satisfactory long-tail service recommendation, most of related approaches revolve around how to enrich the available information of long-tail services. Bai et al. [1] utilized the stacked auto-encoders to extract preference features of developers, and presented a deep learning framework to enrich the usage data of long-tail services. Niu et al. [10] extracted interaction records from the query information of both shop and product, and proposed a dual heterogeneous graph attention network to help target users discover potential interested long-tail shops. Chen et al. [24] treated the displayed and non-displayed services as target domains respectively, and proposed an entire space adaptation approach that utilized correlations between high-level service attributes to accomplish the long-tail service alignment. Li et al. [25] first divided services into short-head and long-tail groups, then, separately executed training on two types of services, and employed an iterative optimization mechanism to produce long-tail service recommendations. Kanat et al. [26] analyzed the relationship between the social effects and network characteristic, and indicated the influence of long-tail market under different social granularity, which could provide strong supports for long-tail service recommendations. Zhou et al. [27] utilized the visual data (e.g., image and video) as side information to enrich the available information, and presented a collaborative training system to recognize long-tail services. Pang et al. [28] transformed the optimal weight vector problem into multi-objective optimization, and proposed a weighted similarity measure according to the non-dominated sorting genetic algorithm to calculate the correlations of long-tail services. However, most of existing long-tail service recommendation approaches focus on extracting embeddings from heterogeneous network (e.g., interaction network), but ignoring the rich feature information in homogeneous network, resulting into the inability to extract multi-aspect embeddings, so as to further affect the recommendation performance in terms of long-tail services.

III. Preliminaries and Problem Statement

In this section, the main notations used in the rest of this research are described. In addition, the problem setting of proposed CTGNN is also discussed.

A. Preliminaries

CTG includes plenty of tagging assignments generated by developers of Web services and mashups. Because tag records not only reflect target mashups' potential requirements but also depict the features of service, we can improve long-tail service recommendation performance by exploring abundant tag information in CTG. Table I summarizes main notations used in this research. Moreover, we give a detailed description about several key notions as follows.

Mashup-Tag-Service Tensor: The tagging assignments can be represented as a three-order tensor $A \subseteq M \times T \times S$, where a tag assignment $a \in A$ can be expressed as $\langle m, t, s \rangle$, and $a = 1$ meaning that mashup m and service s are annotated by the same tag t.

Collaborative Tagging Graph: A collaborative tagging graph can be expressed as an undirected weighted graph $G = (V, E, W)$ with tags as bridge nodes, where V, E, and W represent the sets of nodes, edges, and edge weights, respectively, and $V = M \cup T \cup S$. The edge $\mathbf{e} = (v, v', w) \in E$ indicates the edge weight between nodes v and v' is $w \in W$.

To be specific, in this paper, we divide G into five types of networks as follows.

- $\mathbf{G_{MM}}$: Homogeneous network consists of mashup nodes.
- $\mathbf{G_{SS}}$: Homogeneous network consists of service nodes.
- $\mathbf{G_{MS}}$: Heterogeneous network consists of mashup and service nodes.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF NOTATIONS

Terms	Description
M	Set of mashups: $\{m_1, m_2, \cdots, m_{\|M\|}\}$
S	Set of services: $\{s_1, s_2, \cdots, s_{\|S\|}\}$
T	Set of tags: $\{t_1, t_2, \cdots, t_{\|T\|}\}$
G	Collaborative tagging graph
A	Mashup-tag-service tensor
$\widetilde{M}$	Latent feature matrix of mashups
$\widetilde{S}$	Latent feature matrix of services
$\widetilde{T}$	Latent feature matrix of tags
C	Core tensor of A after decomposition
G_{MM}, G_{SS}	Homogeneous networks extracted from G
G_{MT}, G_{MS}, G_{ST}	Heterogeneous networks extracted from G
e_m	Embedding of mashup m
e_t	Embedding of tag t
e_s	Embedding of service s
$e_m(G_{MM})$	Embedding of mashup m extracted from G_{MM}
$e_s(G_{SS})$	Embedding of service s extracted from G_{SS}
$e_m(G_{MS})$	Embedding of mashup m extracted from G_{MS}
$e_s(G_{MS})$	Embedding of service s extracted from G_{MS}
$e_m(G_{MT})$	Embedding of mashup m extracted from G_{MT}
$e_s(G_{ST})$	Embedding of service s extracted from G_{ST}

- $\mathbf{G_{MT}}$: Heterogeneous network consists of mashup and tag nodes.
- $\mathbf{G_{ST}}$: Heterogeneous network consists of service and tag nodes.

The edge weights in heterogeneous networks can reflect the interaction information between different types of nodes. Edge $\mathbf{e} = (m, s, w_{ms})$ represents the invocation relationship between mashup m and service s, where $w_{ms} = 1$ means mashup m has invoked service s, and 0 otherwise. Edge $\mathbf{e} = (m, t, w_{mt})$ indicates the annotation relationship between mashup m and tag t, where $w_{mt} = 1$ represents mashup m has been annotated by tag t, and 0 otherwise. Edge $\mathbf{e} = (s, t, w_{st})$ expresses service s has been annotated by tag t when $w_{st} = 1$, and 0 otherwise.

B. Problem Statement

Conventional recommendation tasks, such as recommending products or videos, aim to provide personalized recommendations to target users based on their historical interaction data. In these tasks, user interests tend to change gradually. In contrast, service recommendation requires providing timely recommendations to users due to its real-time and adaptive nature. Users may have immediate needs and preferences while creating a mashup, so service recommendation should generate results based on their instant query information. Currently, most existing web service recommendation approaches rely on keyword-based web service search engines, which can be disadvantageous due to their heavy dependence on complex queries from developers. To address this issue, a service RS that recommends potential web services meeting developers' mashup demands by selecting relevant tags would be preferable to the dilemma of using complex and difficult-to-determine keywords.

The main objective of this study is to incorporate CTG into GNNs to recommend as many suitable uninvoked long-tail services as possible for each target mashup (including both constructing and new mashups) based on the target mashup query generated by user-selected tags. Given that CTG comprises numerous tag records that precisely and clearly reflect the mashup requirements and enhance the available information of long-tail services, tag records play a crucial role as a bridge between mashups and long-tail services, streamlining the recommendation of potential long-tail services to target mashups. To be more specific, we define the problem of long-tail service recommendations as follows:

Input: Collaborative tagging graph G and a target mashup query generated by user-selected tags.

Output: The top N services that are most relevant to the target mashup and contain as many long-tail services as possible.

IV. PROPOSED APPROACH

In this section, we formally describe detailed components of CTGNN. Specifically, CTGNN is composed of five core modules. 1) Mashup-tag-service tensor decomposition, which generates low-dimensional representations of mashups, tags, and services for further graph operation. 2) Interaction-aware heterogeneous neighbor aggregation, which extracts embeddings of mashups and services from heterogeneous networks with merging interaction strength. 3) Diffusion-aware homogeneous neighbor aggregation, which calculates the edge weights between nodes in homogeneous networks by heat diffusion principle. 4) Type-aware node embedding updating, which updates embeddings of mashups and services by aggregating three types of embeddings with attention mechanism. 5) Model training, which integrates tensor decomposition task and recommendation task into a jointly learning framework to train the CTGNN.

A. Mashup-Tag-Service Tensor Decomposition

To parameterize nodes of CTG as vector representations, we first extract the mashup-tag-service tensor from CTG. Then, we execute tensor decomposition to produce the latent feature matrices for mashups, tags, and services, respectively. Considering the mashup-tag-service third-order tensor $A \in \mathbb{R}^{|M|\times|T|\times|S|}$, tensor decomposition can decompose the third-order tensor into three low-rank matrices (i.e., $\widetilde{M} \in \mathbb{R}^{|M|\times L}$, $\widetilde{T} \in \mathbb{R}^{|T|\times L}$, and $\widetilde{S} \in \mathbb{R}^{|S|\times L}$) and one core tensor (i.e., $C \in \mathbb{R}^{L\times L\times L}$) [29], [30], where L represents the latent feature dimensions.

Assuming that the decomposed tensor is $\hat{A}$, where

$$\hat{A}_{i,j,k} = \sum_{m=1}^{L}\sum_{t=1}^{L}\sum_{s=1}^{L} C_{m,t,s}\widetilde{M}_{i,m}\widetilde{T}_{j,t}\widetilde{S}_{k,s}. \tag{1}$$

In a real-world CTG, there are plenty of services, however, each mashup invokes only a small number of services. Moreover, there are even fewer cases of mashups and services with the same tags, resulting in a very sparse tensor A. In this case, only a few services with invocations need to be calculated, which easily results into over-fitting problem [29], [30]. Here, we add a regularization term into the loss function:

$$\mathcal{L}_D = \sum_{i=1}^{|M|}\sum_{j=1}^{|T|}\sum_{k=1}^{|S|}\left(A_{i,j,k} - \hat{A}_{i,j,k}\right)^2 + \lambda_1||C||_F^2$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

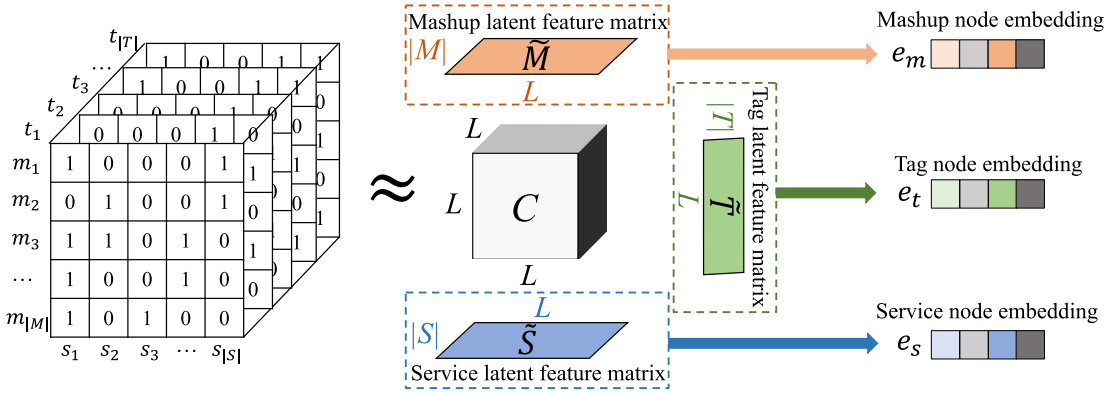


Fig. 3. Process of mashup-tag-service tensor decomposition. The low-rank node embeddings can be obtained from latent feature matrices that are extracted by mashup-tag-service tensor decomposition.

$$+\lambda_2\left(||M||_F^2+||T||_F^2+||S||_F^2\right), \tag{2}$$

where $||\bullet||_F$ means the F-norm. λ_1 and λ_2 indicate the penalty weights of regularization term for loss function. $\mathcal{L}_D$ reflects the differences between the true tag records and decomposed values.

For a given third-order tensor of A, we employ the stochastic gradient descent to continually update the values of C, $\widetilde{M}$, $\widetilde{S}$, and $\widetilde{T}$, respectively. According to the stochastic gradient descent, we train the loss function as follows.

$$\frac{\partial \mathcal{L}_D}{\partial C}=2\sum_{i=1}^{|M|}\sum_{j=1}^{|T|}\sum_{k=1}^{|S|}\left(\hat{A}_{i,j,k}-A_{i,j,k}\right)\widetilde{M}_i\otimes\widetilde{T}_j\otimes\widetilde{S}_k, \tag{3}$$

$$\frac{\partial \mathcal{L}_D}{\partial \widetilde{M}_i}=2\sum_{j=1}^{|T|}\sum_{k=1}^{|S|}\left(\hat{A}_{i,j,k}-A_{i,j,k}\right)C\times_{\widetilde{T}}\widetilde{T}_j\times_{\widetilde{S}}\widetilde{S}_k, \tag{4}$$

$$\frac{\partial \mathcal{L}_D}{\partial \widetilde{T}_j}=2\sum_{i=1}^{|M|}\sum_{k=1}^{|S|}\left(\hat{A}_{i,j,k}-A_{i,j,k}\right)C\times_{\widetilde{M}}\widetilde{M}_i\times_{\widetilde{S}}\widetilde{S}_k, \tag{5}$$

$$\frac{\partial \mathcal{L}_D}{\partial \widetilde{S}_k}=2\sum_{i=1}^{|M|}\sum_{j=1}^{|T|}\left(\hat{A}_{i,j,k}-A_{i,j,k}\right)C\times_{\widetilde{M}}\widetilde{M}_i\times_{\widetilde{T}}\widetilde{T}_j, \tag{6}$$

where $\otimes$ means the tensor product operation. $\times_{\widetilde{M}}$, $\times_{\widetilde{T}}$, and $\times_{\widetilde{S}}$ denote the corresponding mode multiplication. Fig. 3 demonstrates the process of mashup-tag-service tensor decomposition.

B. Interaction-Aware Heterogeneous Neighbor Aggregation

Long-tail services lack exploitable information in CTG, making it difficult to obtain high-quality embeddings and affecting the accuracy of recommendations. Heterogeneous networks, on the other hand, include rich interaction information between different types of nodes, which can provide both neighboring node embedding information and their interaction information for long-tail services. Inspired by this, we propose an interaction-aware heterogeneous neighbor aggregation procedure that integrates the features of the target central node and different types of neighbors, as well as their interaction strengths, to improve the embedding quality for long-tail services and enhance their recommendation accuracy.

For each heterogeneous network, we utilize graph convolutional network to aggregate the features of neighboring nodes to obtain the embedding of central nodes. Given a central mashup node m, the aggregated embeddings from G_{MS} and G_{MT} can be represented by $e_m(G_{MS})$ and $e_m(G_{MT})$, respectively. $e_m(G_{MS})$ and $e_m(G_{MT})$ can be calculated by a weighted sum aggregator with $tanh$ as activation function as follows.

$$e_m(G_{MS})=\tanh\left(\sum_{i\in\{s|w_{ms}=1\}\cup\{m\}}\alpha_{m\leftarrow i}e_i\right), \tag{7}$$

$$e_m(G_{MT})=\tanh\left(\sum_{i\in\{t|w_{mt}=1\}\cup\{m\}}\alpha_{m\leftarrow i}e_i\right), \tag{8}$$

where $\alpha_{m\leftarrow i}$ means the weight assigned to neighbor i for expressing its contribution to the aggregation process. It is worth noting that we add a self-loop operation with weight 1 to each mashup node (i.e., $\alpha_{m\leftarrow m}=1$) for aggregating the embedding of node itself. We also execute this operation for other types of nodes when conducting the convolutional process in the rest of this research.

Fig. 4 gives an example of the process of generating $e_m(G_{MS})$ and $e_m(G_{MT})$ from G_{MS} and G_{MT} through interaction-aware heterogeneous neighbor aggregation for a central mashup node m, where e_s and $e_{w_{ms}}$ denote the embeddings of neighboring services of the mashup m and their edge weight, respectively. e_t and $e_{w_{mt}}$ represent the embeddings of neighboring tags of the mashup m and their edge weights, respectively. Similarly, for a central service node s, we can gain the aggregated embeddings $e_s(G_{MS})$ and $e_s(G_{ST})$ from two heterogeneous networks (i.e., G_{MS} and G_{ST}) as follows.

$$e_s(G_{MS})=\tanh\left(\sum_{i\in\{m|w_{ms}=1\}\cup\{s\}}\alpha_{s\leftarrow i}e_i\right), \tag{9}$$

$$e_s(G_{ST})=\tanh\left(\sum_{i\in\{t|w_{st}=1\}\cup\{s\}}\alpha_{s\leftarrow i}e_i\right), \tag{10}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

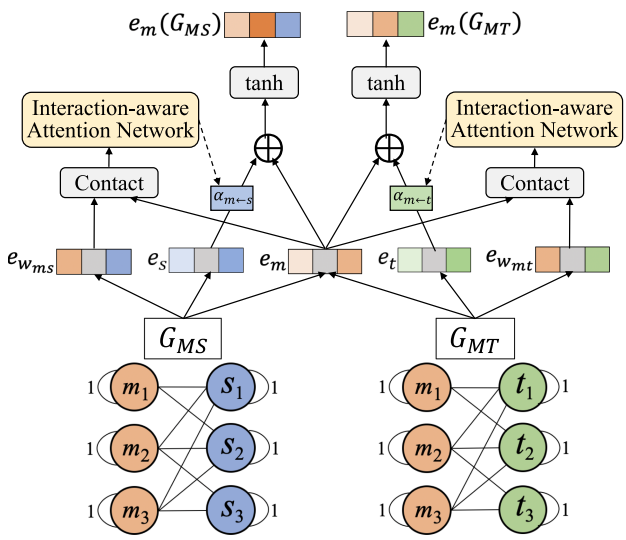


Fig. 4. Basic process of generating a target mashup m's multi-aspect embeddings (i.e., $e_m(G_{MS})$ and $e_m(G_{MT})$) from heterogeneous networks (i.e., G_{MS} and G_{MT}) through interaction-aware heterogeneous neighbor aggregation.

where $\alpha_{s\leftarrow i}$ indicates the weight of neighbor i when aggregating it to the central node s. To calculate the weight (e.g., $\alpha_{m\leftarrow i}$ or $\alpha_{s\leftarrow i}$), one of the most common approaches is to set it as the reciprocal of the sum number of adjacent nodes. However, it will cause all neighboring nodes to have the same weight, which is obviously not reasonable in practical applications. In order to alleviate this problem, we utilize the attention mechanism which integrates the embeddings of both node itself and interaction to determine the weight value. Here, we take the weight $\alpha_{m\leftarrow i}$ in (7) as an example. First, we compute the pre-weight $\alpha'_{m\leftarrow i}$, (11) describes the weight calculation process.

$$\alpha'_{m\leftarrow i} = \mathrm{ReLU}(W_1^{\alpha}[e_m||e_{w_{mi}}] + W_2^{\alpha}e_i + b^{\alpha}),$$
$$s.t.\ \ i \in \{s|w_{ms} = 1\} \cup \{m\}, \tag{11}$$

where $\|$ denotes the concatenation operation, e_m and e_i represent the embedding of central mashup node m and neighboring node i. $e_{w_{mi}}$ is the embedding of edge weight w_{mi}. $ReLU$ indicates the activation function of the neural attention layer. W_1^{α}, W_2^{α}, and b^{α} represent the trainable parameters of attention network. Furthermore, the final attention weight $\alpha_{m\leftarrow i}$ can be gained through executing $softmax$ function on the calculated attentive score, i.e., $\alpha_{m\leftarrow i} = softmax(\alpha'_{m\leftarrow i})$.

C. Diffusion-Aware Homogeneous Neighbor Aggregation

Homogeneous network consists of nodes with the same type, the weights between homogeneous nodes can reflect similarities or correlations between mashups or services. Supposing that similar mashups/services also own similar feature vectors, GNNs perform graph convolution operation on homogeneous nodes to smooth the feature signal. However, homogeneous networks (e.g., G_{MM} and G_{SS}) cannot be constructed from CTG directly, because there does not exist straightforward edges between homogeneous nodes in CTG. Therefore, to execute GNNs on homogeneous networks, we should first construct edges as well as edge weights for homogeneous nodes.

Generally, for a CTG, if we simply regard the number of times that two services have been annotated by the same tag as the edge weights, we can construct a weighted service homogeneous network easily. By the same principle, a mashup homogeneous network can also be established directly. However, because the extreme sparsity of CTG, the weight matrices in two homogeneous networks will also be very sparse, resulting into that the feature information of neighboring nodes in homogeneous network cannot be aggregated effectively. To this end, we propose a diffusion-aware homogeneous neighbor aggregation to assign edge weight for each pair of nodes, and further aggregate feature information of homogeneous neighboring nodes to the central node.

Given a central service node s, the aggregated neighboring embedding of node s from G_{SS} can be represented by $e_s(G_{SS})$. Similarly, $e_m(G_{MM})$ for a central mashup node m. Equations (12) and (13) show the aggregation operations for $e_s(G_{SS})$ and $e_m(G_{MM})$ respectively.

$$e_s(G_{SS}) = \tanh\left(\sum_{s'\in N_s\cup\{s\}} \beta_{s\leftarrow s'}e_{s'}\right), \tag{12}$$

$$e_m(G_{MM}) = \tanh\left(\sum_{m'\in N_m\cup\{m\}} \beta_{m\leftarrow m'}e_{m'}\right), \tag{13}$$

where $\beta_{s\leftarrow s'}$ and $\beta_{m\leftarrow m'}$ are the weights assigned to neighboring node s' and m' when executing aggregation operation on G_{SS} and G_{MM} respectively. N_s and N_m represent the set of neighboring nodes of the central node s and m respectively.

In order to assign $\beta_{s\leftarrow s'}$ and $\beta_{m\leftarrow m'}$ in two homogeneous networks, we utilize a diffusion-aware weight assignment which employs head diffusion principle to generate edge weights for each pair of services/mashups. head diffusion was originally used to indicate a physical phenomenon, which has been proved to be able to generate diverse recommendations in the data sparse environment [31], thus it has been widely utilized into RSs in recent years [32]. Because head diffusion allocates more energy to "cold" nodes (e.g., long-tail nodes) than "hot" nodes (e.g., short-head nodes), we can employ this characteristic to assign higher weights to long-tail nodes during the process of convolution. head diffusion supposes each target node has an initial energy, and the target node distributes its energy to neighboring nodes through a bipartite network based resource-allocation process. Then, each node further passes back its received energy to neighboring nodes. The final received energy can reflect the correlation between the central node and other nodes.

Here, we utilize the construction of service homogeneous network as an example, and give the process of calculating the edge weight between a central service with other services. The diffusion-aware weight calculation mainly contains three steps. First, if we treat service node s_i as the central node, an initial energy $f(s_i) > 0$ will be allocated to it, and the initial energy of other nodes is assigned to 0. Next, the energy diffuses from service nodes to tag and mashup nodes respectively. Service node

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

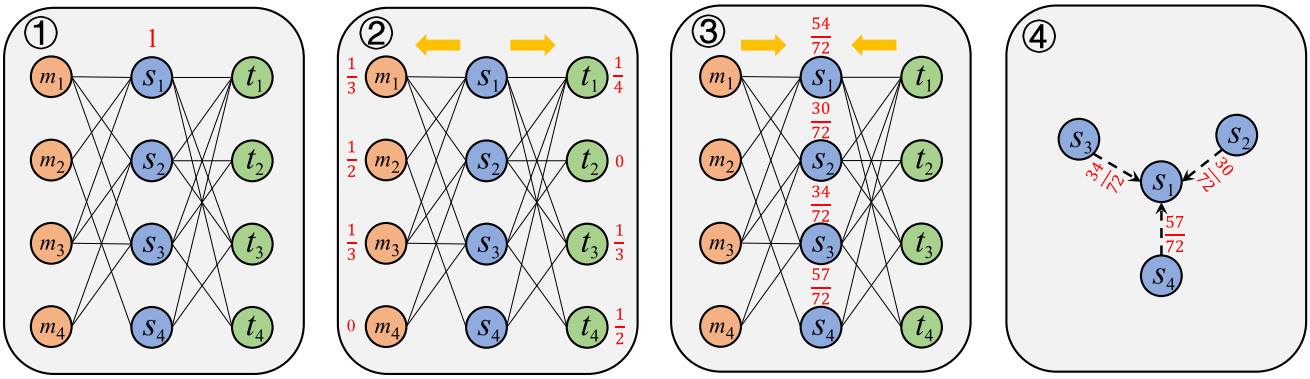


Fig. 5. Basic flow of diffusion-aware weight computation for service homogeneous network. Here, s_1 is a central service node that is allocated the initial energy $f(s_1) = 1$. s_4 is a long-tail service node that is invoked only once.

s_i allocates energy to mashups and tags according to the node degrees. The energies $f(m_x)$ and $f(t_y)$ received by mashups and tags are the sum of energies collected from connected services. Equations (14) and (15) formulate the energy that diffuses from service nodes to mashup and tag nodes respectively.

$$f(m_x) = \sum_{i=1}^{|S|} \frac{w_{s_i m_x} f(s_i)}{D(m_x)}, \tag{14}$$

$$f(t_y) = \sum_{i=1}^{|S|} \frac{w_{s_i t_y} f(s_i)}{D(t_y)}, \tag{15}$$

where $D(m_x)$ and $D(t_y)$ represent the degrees of nodes m_x and t_y. $w_{s_i m_x}$ means the edge weight between service s_i and mashup m_x, and $w_{s_i t_y}$ indicates the edge weight between service s_i and tag t_y. Finally, the energies are rerouted from mashup and tag nodes to service nodes respectively. The total energy $f(s_j)$ that service node s_j receives is the sum of energy diffused from connected mashup nodes and tag nodes. Equation (16) demonstrates the proportion of node s_i's energy allocated to s_j, where $D(s_i)$ represents the degree of node s_i, respectively. It is clear that the more energy a service node receives from another service node, the closer the relationship between them, and vice versa.

$$\begin{aligned} f(s_j) &= \sum_{x=1}^{|M|} \frac{w_{s_j m_x} f(m_x)}{D(s_j)} + \sum_{y=1}^{|T|} \frac{w_{s_j t_y} f(t_y)}{D(s_j)} \\ &= \sum_{x=1}^{|M|} \frac{w_{s_j m_x}}{D(s_j)} \sum_{i=1}^{|S|} \frac{w_{s_i m_x} f(s_i)}{D(m_x)} + \sum_{y=1}^{|T|} \frac{w_{s_j t_y}}{D(s_j)} \sum_{i=1}^{|S|} \frac{w_{s_i t_y} f(s_i)}{D(t_y)} \\ &= \sum_{i=1}^{|S|} \left(\sum_{x=1}^{|M|} \frac{w_{s_j m_x} w_{s_i m_x}}{D(m_x) D(s_j)} + \sum_{y=1}^{|T|} \frac{w_{s_j t_y} w_{s_i t_y}}{D(t_y) D(s_j)} \right) f(s_i). \end{aligned} \tag{16}$$

In this paper, we utilize the proportion of energy diffused from node s_i to node s_j to represent the edge weight $\beta_{s_i \leftarrow s_j}$ as

$$\beta_{s_i \leftarrow s_j} = \sum_{x=1}^{|M|} \frac{w_{s_j m_x} w_{s_i m_x}}{D(m_x) D(s_j)} + \sum_{y=1}^{|T|} \frac{w_{s_j t_y} w_{s_i t_y}}{D(t_y) D(s_j)}. \tag{17}$$

From the head diffusion process, we can clearly find that the diffusion matrix is asymmetric, meaning that $\beta_{s_j \leftarrow s_i}$ is no equal to $\beta_{s_i \leftarrow s_j}$, which can ensure that different nodes have individual and different aggregation process. Fig. 5 gives an example of the process of diffusion-aware weight computation for service homogeneous network. As can be seen in the figure, supposing service node s_1 as the central service node, we assign an initial energy $f(s_1) = 1$ to node s_1. Since service node s_4 is invoked only once, which can be considered as long-tail service. Then, the energy of service node s_1 diffuses toward mashups and tags, respectively. According to (14), we can obtain $f(m_1) = 1/3$, $f(m_2) = 1/2$, $f(m_3) = 1/3$, and $f(m_4) = 0$. On the other hand, we can gain $f(t_1) = 1/4$, $f(t_2) = 0$, $f(t_3) = 1/3$, and $f(t_4) = 1/2$ based on (15). Finally, the energy diffuses from mashup/tag nodes back to service nodes. We can obtain $\beta_{s_1 \leftarrow s_2} = 30/72$, $\beta_{s_1 \leftarrow s_3} = 34/72$, and $\beta_{s_1 \leftarrow s_4} = 57/72$ according to (17). From the head diffusion process in Fig. 5, we can conclude that the long-tail service (e.g., s_4) will be assigned a higher weight during the neighbor aggregation, meaning that the central service will aggregate more feature information from long-tail services.

D. Type-Aware Node Embedding Updating

After executing aggregation operations on both heterogeneous and homogeneous networks, three types of embeddings will be generated for mashups and services respectively, i.e., $e_m(G_{MM})$, $e_m(G_{MS})$, and $e_m(G_{MT})$ for mashups, $e_s(G_{SS})$, $e_s(G_{MS})$, and $e_s(G_{ST})$ for services. Then, we merge three type-specific embeddings and update node information for mashups and services respectively.

Clearly, different types of embeddings have different emphasis on expressing the feature of nodes. Here, for the purpose of extracting efficient type embeddings adaptively, we propose a type-aware attention network to re-scale the weights of type-specific embeddings and enhance the role of more efficient

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

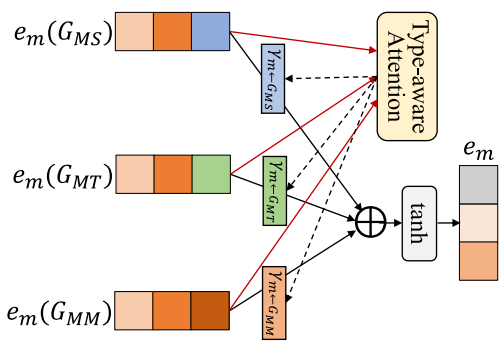


Fig. 6. Process of type-aware embedding updating for mashup node e_m.

embeddings. First, we utilize a pooling-based function to integrate and update node embeddings for mashups and services respectively, shown as

$$e_m = \tanh\left(\sum_{i\in\{MM\}\cup\{MS\}\cup\{MT\}} \gamma_{m\leftarrow G_i} e_m(G_i)\right), \quad (18)$$

$$e_s = \tanh\left(\sum_{i\in\{SS\}\cup\{MS\}\cup\{ST\}} \gamma_{s\leftarrow G_i} e_s(G_i)\right), \quad (19)$$

where $\gamma_{m\leftarrow G_i}$ and $\gamma_{s\leftarrow G_i}$ indicate the weights assigned to embeddings $e_m(G_i)$ and $e_s(G_i)$ respectively. Then, we utilize a two-layer attention network to determine the value of $\gamma_{m\leftarrow G_i}$ and $\gamma_{s\leftarrow G_i}$. Equations (20) and (21) formulate the process.

$$\gamma_{m\leftarrow G_i} = \mathrm{ReLU}\left(W_1^{\gamma} e_m(G_i) + b_1^{\gamma}\right), \quad s.t.\ i \in \{MM\}\cup\{MS\}\cup\{MT\}, \quad (20)$$

$$\gamma_{s\leftarrow G_i} = \mathrm{ReLU}\left(W_2^{\gamma} e_s(G_i) + b_2^{\gamma}\right), \quad s.t.\ i \in \{SS\}\cup\{MS\}\cup\{ST\}, \quad (21)$$

where $W_1^{\gamma}, W_2^{\gamma}, b_1^{\gamma}$, and b_2^{γ} are the trainable parameters. Similar to $\alpha_{m\leftarrow s}$, the weight $\gamma_{m\leftarrow G_i}$ and $\gamma_{s\leftarrow G_i}$ can be gained by normalizing the above attentive scores by $softmax$ function. Fig 6 illustrates the process of type-aware embedding updating for the mashup node e_m.

Finally, we adopt the inner product for the embeddings of mashups and services to predict scores (shown in (22)). And top N services with highest predicted scores will be recommended to the target mashup.

$$\hat{y}_{ms} = e_m^{\top} e_s. \quad (22)$$

E. Model Training

In order to learn target mashups' requirements on long-tail services, we employ Bayesian Personalized Ranking (BPR) as our loss function which has been popularly utilized in recommendation tasks. The BPR loss induces the model to rank the positive samples (invoked services) higher than negative samples (uninvoked services). On the other hand, our proposed CTGNN aims to generate top N service recommendations rather than rating score prediction, which is just in line with BPR loss function that is more effective to generate service ranking. The BPR loss function is as follows.

$$\mathcal{L}_{BPR} = \sum_{\langle m, s_+, s_-\rangle} -\ln sigmoid\left(\hat{y}_{ms_+} - \hat{y}_{ms_-}\right), \quad (23)$$

where s_+ and s_- mean the positive (invoked) and negative (uninvoked) services of mashup m respectively.

Finally, to train the parameters more efficiently, we integrate the tensor decomposition task and recommendation task to form the final loss function as follows.

$$\mathcal{L} = \mathcal{L}_D + \mathcal{L}_{BPR} + \lambda||\theta||_2, \quad (24)$$

where λ indicates the regularization weight that controls the penalty strength, and θ represents model parameters, respectively. We employ mini-batch strategy to optimize $\mathcal{L}_D$ and $\mathcal{L}_{BPR}$ alternatively. In addition, dropout mechanism is also utilized to alleviate over-fitting problem. Particularly, a mini-batch of positive invocation (m, s_+) is first sampled. Furthermore, for each positive invocation, negative (uninvoked) services are randomly sampled with a uniform distribution and negative invocation (m, s_-) is structured. Finally, we utilize both positive and negative invocations to train model and minimize the loss function.

F. Computational Complexity Analysis

Here, we formally analyze the time complexity of proposed CTGNN according to its four main steps.

- *Mashup-tag-service tensor decomposition:* The time complexity of tensor decomposition is determined by the number of observations in the tensor and the eigenvector dimensions of mashups, tags, and services. It can be expressed as $\mathcal{O}(|M||T||S|L^3)$.
- *Interaction-aware heterogeneous neighbor aggregation:* This step involves performing operations on three different types of heterogeneous networks. The time complexities of these operations for G_{MS}, G_{MT}, and G_{ST} are $\mathcal{O}((|M|+|S|)L^2 + E_{MS}L)$, $\mathcal{O}((|M|+|T|)L^2 + E_{MT}L)$, and $\mathcal{O}((|S|+|T|)L^2 + E_{ST}L)$, respectively. Since these operations can be executed independently and in parallel, and considering the relative sizes of the variables involved, the total time complexity of this step is approximately $\mathcal{O}((|M|+|S|)L^2 + E_{MS}L)$.
- *Diffusion-aware homogeneous neighbor aggregation:* This step includes diffusion-aware weight assignment and neighbor aggregation. The time complexities of diffusion-aware weight assignment for mashups and services are $\mathcal{O}(|M||S| + |T|(|M|+|S|))$. The neighbor aggregation process on G_{MM} and G_{SS} has time complexities of $\mathcal{O}(|M|L^2)$ and $\mathcal{O}(|S|L^2)$, respectively. The overall time complexity of this step is approximately $\mathcal{O}(|M||S| + |T|(|M|+|S|))$.
- *Type-aware node embedding updating:* This step involves updating node embeddings for both mashups and services, resulting in a time complexity of $\mathcal{O}(|M|L^2 + |S|L^2)$.

Based on the analysis above, the total time complexity of CTGNN is $\mathcal{O}(|M||T||S|L^3 + |T|(|M|+|S|))$, which can be

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTICAL RESULTS OF TWO DATASETS

Parameters	ProWeb2014	ProWeb2020
Number of Web APIs	13,384	16,248
Number of mashups	6,825	7,475
Number of tags	3,126	3,358
Number of edges between Web APIs and mashups	18,291	20,332
Average number of Web APIs per mashup	2.68	2.72
Average number of tags per Web API	3.12	3.24
Average number of tags per mashup	3.38	3.51

further simplified to $\mathcal{O}(|M||T||S|L^3)$. Therefore, it can be concluded that the proposed CTGNN model has the same level of time complexity as tensor decomposition without adding a large amount of extra time consumption.

V. EXPERIMENTS AND EVALUATIONS

In this section, first, we give a detailed introduction about experiment setup that contains the experimental datasets, validation process, and evaluated metrics. Second, in order to maintain the best performance, we extract the optimal parameters for our proposed CTGNN. Then, comparative experiments are executed with the state-of-the-art approaches to express the superior performance of CTGNN. Next, we analyze the impact of dynamic characteristic of CTG on the recommendation performance. Finally, ablation study is executed to demonstrate the effects of different components of CTGNN.

A. Experimental Setup

To evaluate the performance of CTGNN, we utilize two real-world service datasets crawled from ProgrammableWeb, the largest online repository of public Web services that has been widely used for Web service analysis and recommendation [1], [3], [18]. The first dataset, ProWeb2014, was crawled from ProgrammableWeb.com from June 2005 to December 2014, and contains 13,384 Web APIs, 6,825 mashups, and 3,126 tags. There are 18,291 edges between Web APIs and mashups in this dataset. The second dataset, ProWeb2020, was crawled from ProgrammableWeb.com from September 2018 to December 2020, and contains 16,248 Web APIs, 7,475 mashups, and 3,358 tags. There are 20,332 edges between Web APIs and mashups in this dataset. Each Web API and mashup is associated with several functional tags. We constructed the CTG by using the observed link relationships between mashups, tags, and Web APIs. The statistical results of two datasets are presented in Table II.

In the experiments, to validate the performance of our proposed approach, we used the validation approach proposed in [33] to generate training and testing sets. Specifically, we divide the dataset into training sets and testing sets using a moving cutoff timestamp. We set the time granularity as one month, and the length of the cutoff timestamp is also one month. We use the data before the cutoff timestamp as the training set, and the data with exactly that cutoff timestamp as testing set. We obtain

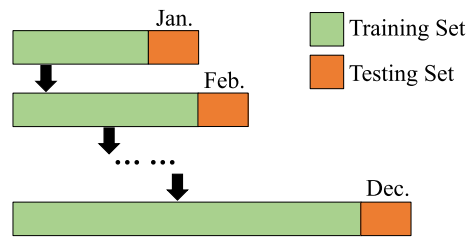


Fig. 7. Illustration of generating training and testing datasets.

twelve corresponding training datasets and testing datasets for both ProWeb2014 and ProWeb2020 datasets through setting the cutoff timestamp from January to December of 2014 and 2020 respectively. Fig. 7 gives an illustration of generating training and testing datasets. It is worth noting that, to ensure effective convolutional representation learning and enhance experimental credibility by accurately reflecting real-world applications, we removed isolated services from both datasets. These isolated services, which refer to Web APIs that lacked connections to any mashups or tags, were eliminated after dividing the datasets into training and testing sets based on the cutoff timestamp. Specifically, in the ProWeb2014 dataset, a total of 420 isolated Web APIs were removed during preprocessing, while in the ProWeb2020 dataset, 705 isolated Web APIs were removed.

For each mashup that appears in the testing datasets, we utilize its composite services as the ground-truth recommendation. We calculate the experimental results obtained from different testing sets and use their average as the final result. It is worth noting that, similar with research [1], we treat services that have been invoked no more than 5 times as long-tail services. This is because such services account for nearly 20% of the total service consumption by mashups, which is consistent with the characteristics of long-tail services described in Section I. In addition, we employ the tags connected with the target mashup as user-selected tags to represent the target mashup query.

To further evaluate the proposed CTGNN, we employ precision and recall metrics to evaluate the recommendation accuracy [34], [35]. In addition, we utilize mean diversity (MD) and long-tail ratio (LTR) to evaluate the recommendation diversity [36]. Specially, LTR can further reflect the model ability of discovering and recommending the long-tail services [1].

The precision refers to the proportion of the number of recommended services invoked by a mashup in the whole recommended services. It reflects the probability that the mashup has the requirement among recommended services. The recall indicates the proportion of the number of recommended services invoked by a mashup in all the invoked services. It reflects the probability that a service invoked by mashup will be recommended. It is clear that the larger values of three metrics indicate the higher recommendation accuracy. Assuming that $Rec_{m_i}(N)$ represents the set of N number of recommended services for mashup m_i, and $H(m_i)$ denotes the set of services that mashup m_i actually invoked. The definitions of precision and recall metrics can be described as follows.

$$\text{Precision} = \frac{\sum_{i=1}^{|M|} |Rec_{m_i}(N) \cap H(m_i)|}{\sum_{i=1}^{|M|} |Rec_{m_i}(N)|}, \tag{25}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

$$\text{Recall} = \frac{\sum_{i=1}^{|M|} |Rec_{m_i}(N) \cap H(m_i)|}{\sum_{i=1}^{|M|} |H(m_i)|}. \tag{26}$$

The MD metric denotes the average value of overlap degree between each pair of mashups' recommended services. Assuming there are two mashups m_i and m_j, we extract top N number of recommendations $Rec_{m_i}(N)$ and $Rec_{m_j}(N)$, and further normalize it to obtain the overlap degree between two mashups' recommendation list. It is obvious that an approach owning higher recommendation diversity will have a larger value of MD, vice versa.

$$\text{MD} = 1 - \frac{1}{N}\frac{2}{|M|(|M|-1)} \sum_{1 \le i < j \le |M|} |Rec_{m_i}(N) \cap Rec_{m_j}(N)|. \tag{27}$$

The LTR metric refers to the proportion of the number of recommended long-tail services in the whole set of long-tail services, which reflects the ability of model to recommend long-tail service. If we suppose LST as the set of long-tail services, the LTR metric can be defined as follows.

$$\text{LTR} = \frac{\sum_{i=1}^{|M|} |Rec_{m_i}(N) \cap LTS|}{|LST|}. \tag{28}$$

B. Parameter Settings

To train our proposed CTGNN, we first utilize the random normal distribution to initial all of the weight matrices, and treat all the biases as zero vectors. Then, we optimize the loss function using mini-batch Adagrad since Adagrad owns good performance on processing sparse data. Besides, the Adagrad can efficiently generate optimal solution by automatically adjusting the learning rate during training. For example, Adagrad employs a higher learning rate to update the lower frequency parameters, while utilizing a lower learning rate to update the higher frequency parameters. To balance the training effect and calculation cost, we set the number of latent dimension L as 64, and the batch size as 256. To alleviate the over-fitting problem, the dropout ratio is set as 0.5. We set the embedding size of nodes and attention factor to 64 and 32 respectively, and the embedding size of edge weights to 10. In addition, we utilize three hidden layers to form a tower pattern, while the dimension from bottom to top is set to 256, 128, and 64, respectively. The maximum number of iterations is fixed to 400. The model training will be terminated when the CTGNN converges or reaches the maximum number of iterations. To show the trend of evaluation metrics with the number of recommendations increases, the number of recommendation was determined as $N \in \{5, 10, 15, 20, 25, 30\}$ in the rest of our experiments [37].

C. Performance Comparisons

To further demonstrate the performance of our proposed CTGNN on long-tail service recommendation, we compare it with the state-of-the-art approaches on real-world ProgrammableWeb dataset.

- ***IBCF*** *(item-based collaborative filtering) [38]:* IBCF is a baseline of collaborative filtering that generates recommendations according to the information of neighborhood of target nodes.
- ***FM*** *(factorization machines) [39]:* FM is a baseline of context-aware recommendation approach that can effectively estimate parameters and produce satisfying recommendations under sparse data environment.
- ***DBRA*** *(diffusion based recommendation algorithm) [31]:* DBRA is a diffusion-based recommendation approach that integrates head diffusion into recommendation process, and employs both user-service interactions and collaborative tagging information to produce service recommendation.
- ***AIRec*** *(attentive intersection recommendation) [13]:* AIRec is a state-of-the-art GNN-based recommendation approach which utilizes a hierarchical attention network to extract the representations of users, tags, and services, and concatenates representations using factorization machines to generate recommendations.
- ***TGCN*** *(tag graph convolutional network) [12]:* TGCN is a state-of-the-art tag-aware recommendation approach which utilizes the contextual semantics of multi-hop neighboring nodes in CTG to generate satisfying recommendations.

In order to fairly make comparisons with approaches mentioned above, a grid search strategy was employed to select the optimal hyper-parameters for each approach. The best performance of IBCF was gained with the number of neighbor set to 25. FM achieved the best performance with the factorization dimensionality set to 8. For DBRA, the best performance appeared when the tunable parameter was 0.4. The optimal solution of AIRec was produced when the size of latent factor was set to 8, and the dropout ratio was set to 0.5. TGCN owned the best performance when the embedding size of nodes and attention factor was 64 and 32 respectively.

Figs. 8 and 9 demonstrate the experimental results of accuracy metrics (i.e., precision and recall) and diversity metrics (i.e., MD and LTR) on ProWeb2014 and ProWeb2020 datasets respectively. As shown in the figure, the results of precision decrease with increasing the number of recommendation, it is because the proportion of services that target mashup actually invoked decreases as the number of recommendation increases. AIRec performs better than FM since AIRec takes both explicit and implicit tags information and utilizes the hierarchical attention network to extract multi-aspect representations accurately. TGCN owns the second best performance over all ranges of recommendation. However, our proposed CTGNN achieves higher values than other approaches. Even compared to the closest competitor TGCN, CTGNN is always superior to TGCN with increasing the number of recommendation. The results of recall metric increase as the number of recommendation increases, it is because the number of services that the target mashup actually invoked is fixed, in this case, the number of services that appear in both recommendation list and the actually invoked set will increase as the number of recommendation increases. The IBCF and DBRA approaches are lower than

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

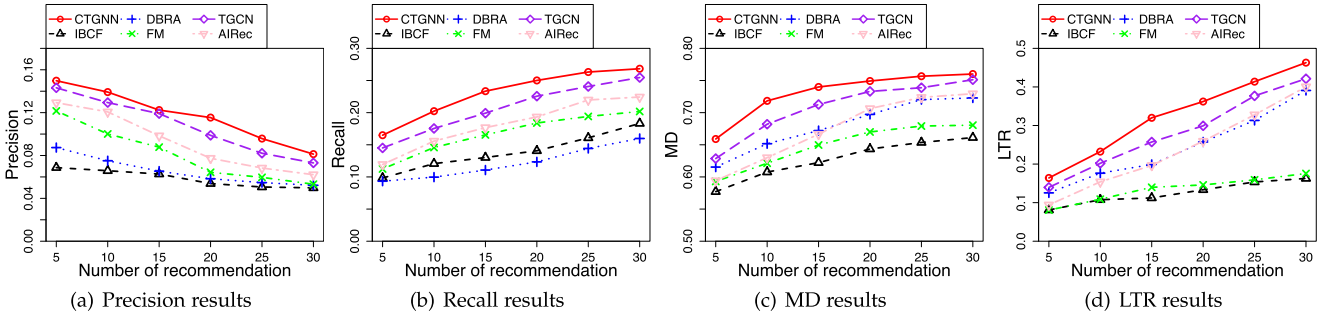


Fig. 8. Comparison results of accuracy metrics (a) precision and (b) recall, and diversity metrics (c) MD and (d) LTR on ProWeb2014 dataset.

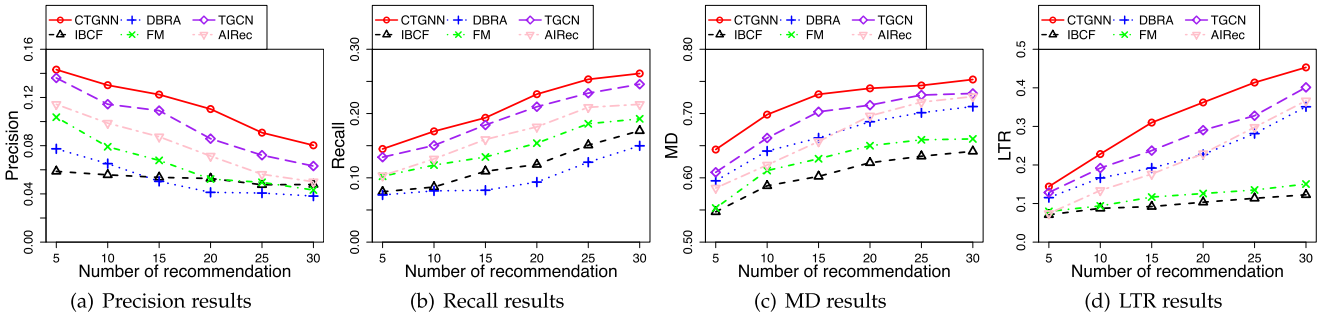


Fig. 9. Comparison results of accuracy metrics (a) precision and (b) recall, and diversity metrics (c) MD and (d) LTR on ProWeb2020 dataset.

other approaches, and DBRA owns the worst performance as the number of recommendation increases. Although TGCN seems to be the closest competitor with the proposed CTGNN, our proposed CTGNN owns the highest values over all ranges of recommendation. This is because CTGNN relies on the powerful representational capacity of GNN and attention mechanism, and further merges features from both heterogeneous and homogeneous networks into the embeddings of mashups and services. Since the higher the values of precision and recall, the better the recommendation accuracy, indicating that the proposed CTGNN generates superior performance than the state-of-the-art approaches in the aspect of recommendation accuracy, and can recommend more accurate services for target mashup.

On the other hand, different from the performance of accuracy metrics, the values of diversity metrics increase with increasing the number of recommendation. For both of the MD and LTR metrics, the IBCF approach owns the lowest values over all the range of recommendations. DBRA is higher than IBCF and FM since DBRA applies head diffusion into recommendation process. AIRec is lower than DBRA when the number of recommendation is less than 15; however, AIRec increases faster than DBRA, and obtains higher values when the number of recommendation is more than 20. Our proposed CTGNN always performs best even comparing with the closest competitor TGCN, it is because CTGNN executes the diffusion-ware homogeneous neighbor aggregation before generating recommendations. Because the higher values of MD and LTR, the better the recommendation diversity, meaning that the proposed CTGNN performs better at generating diverse recommendations than state-of-the-art approaches, especially for the long-tail service recommendations.

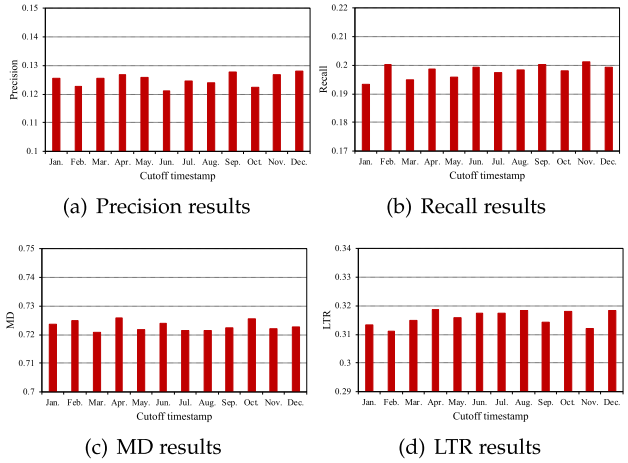
D. Impact of Dynamic CTG

CTG is a dynamic graph as tag information in CTG is continuously updated by developers or managers over time. In this study, we examine the impact of the dynamic nature of CTG on the performance of the proposed CTGNN for recommendation. Specifically, we analyze the individual testing set (i.e., individual months) results in the two datasets based on the experimental results presented in Section V-C, to simulate the dynamic performance of CTG. We then calculate the mean of each month in the two datasets as the final experimental results. Fig. 10 shows the recommendation quality at different cutoff timestamps under the dynamic CTG. As depicted in the figure, the highest and lowest values of accuracy and diversity metrics occur in different months. For instance, the highest value of precision is recorded in September, while November for the recall metric. However, the difference between their highest and lowest values is not significant. From the experimental results, it can be inferred that the dynamic nature of CTG has a minor effect on the recommendation quality of the proposed CTGNN.

E. Ablation Study

To further analyze the effects of three core components in CTGNN, we retain only one component individually in CTGNN to produce three simplified versions, respectively. Then, we

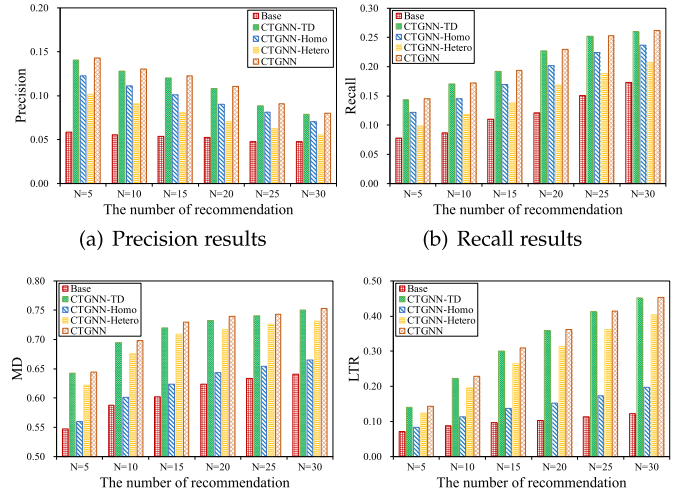
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.



(a) Precision results (b) Recall results

(c) MD results (d) LTR results

Fig. 10. Experimental values under the dynamic CTG on accuracy metrics (a) precision and (b) recall, and diversity metrics (c) MD and (d) LTR.



(a) Precision results (b) Recall results

(c) MD results (d) LTR results

Fig. 11. Experimental results of ablation study on accuracy metrics (a) precision and (b) recall, and diversity metrics (c) MD and (d) LTR on ProWeb2020 dataset.

compare CTGNN with three variations as well as the base version.

- *Base:* A base version that only utilizes mashup-tag-service tensor decomposition to generate embeddings and employs inner product to generate service recommendation.
- *CTGNN-TD:* A simplified version that utilizes random initialization to generate initial embeddings instead of tensor decomposition, while the remaining procedures are the same as those in CTGNN.
- *CTGNN-Hetero:* A simplified version that only retains the diffusion-aware homogeneous neighbor aggregation after mashup-tag-service tensor decomposition and updates the node embedding without utilizing the embeddings extracted from heterogeneous network.
- *CTGNN-Homo:* A simplified version that only employs the interaction-aware heterogeneous neighbor aggregation after mashup-tag-service tensor decomposition and updates the node embedding without utilizing the embeddings extracted from homogeneous network.

Given the consistent performance of the ablation study on both the ProWeb2014 and ProWeb 2020 datasets, we only discuss the results on the ProWeb2020 dataset here. Fig. 11 reports the results, which reveal that the Base approach had the lowest values in terms of both accuracy and diversity metrics, indicating that only utilizing tensor decomposition cannot produce satisfying long-tail service recommendations. On the other hand, CTGNN-TD had almost the same performance as CTGNN, indicating that tensor decomposition cannot improve the performance of CTGNN. However, we found that CTGNN-TD required more running time while training the model to achieve nearly the same optimal performance as CTGNN. Specifically, CTGNN required 187 minutes, while CTGNN-TD required 476 minutes.

Both CTGNN-Hetero and CTGNN-Homo outperformed the Base approach in terms of both accuracy and diversity metrics, indicating that both the diffusion-aware homogeneous neighbor aggregation and interaction-aware heterogeneous neighbor aggregation can enhance the recommendation quality. CTGNN-Homo had higher values than CTGNN-Hetero for recommendation accuracy, indicating that interaction-aware heterogeneous neighbor aggregation played a greater role in improving the recommendation accuracy. This is because interaction-aware aggregation can aggregate multi-type neighboring nodes' embeddings and their interaction information into the target node, resulting in higher quality embedding than diffusion-aware aggregation, which only aggregates the embeddings from same type of nodes (i.e., homogeneous network). However, CTGNN-Hetero performed better than CTGNN-Homo for recommendation diversity, indicating that diffusion-aware aggregation has a more significant effect on enhancing the recommendation diversity. This is because diffusion-aware homogeneous neighbor aggregation can allocate higher weight to long-tail services, allowing the embedding of the target node to absorb more information from long-tail services when executing the aggregation process. In conclusion, the proposed CTGNN outperformed the other approaches in terms of both recommendation accuracy and diversity.

Based on the results of the ablation study, we concluded that tensor decomposition can accelerate the model optimization to the optimal speed. Furthermore, diffusion-aware homogeneous neighbor aggregation plays a greater role in improving the recommendation diversity, while interaction-aware heterogeneous neighbor aggregation contributes more to enhance the recommendation accuracy.

F. Case Study

To gain deeper insights into CTGNN and other compared approaches, we conducted a case study using the $Housing\ Lookup$ mashup from ProgrammableWeb.com. This mashup provides rental and housing information and is tagged with $Real\ estate$, $Mapping$, and $Rentals$. It utilizes four types of APIs, including two long-tail APIs, $GeoAdmin$ and $Mapstraction$,

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EXAMPLE MASHUP AND THE RANK POSITION OF RECOMMENDED APIS GIVEN BY EACH APPROACH

Mashup	*Housing Lookup*					
User-selected tags	*Real estate*; *Mapping*; *Rentals*					
APIs	Rank position of recommended APIs					
	CTGNN	IBCF	FM	DBRA	AIRec	TGCN
GeoAdmin (#Invocations=1)	3	75	42	34	27	12
Mapstraction (#Invocations=2)	7	43	31	22	19	15
MapQuest (#Invocations=8)	19	23	20	25	21	17
Mapbox (#Invocations=11)	24	21	28	26	34	27

and two short-head APIs, $MapQuest$ and $Mapbox$. We can find detailed results in Table III, as shown in the table, CTGNN outperforms other approaches significantly when recommending long-tail services with few previous innovations. For example, the rank position of $GeoAdmin$ is third for CTGNN, which is significantly better than the results of IBCF and FM, which are 75th and 42nd, respectively. However, for short-head services with sufficient previous innovations, CTGNN performs at least as well as other approaches. For instance, the rank position of $Mapbox$ is 24th for CTGNN, which is almost the same as the results of IBCF and DBRA, which are 21st and 26th, respectively. This is because CTGNN leverages both diffusion-aware aggregation and interaction-aware aggregation in the recommendation process to capture better long-tail service representations. In doing so, CTGNN can meet mainstream API requirements of developers while exploring long-tail APIs as much as possible.

VI. CONCLUSIONS AND FUTURE WORK

In this article, we propose a novel CTGNN to generate satisfying long-tail service recommendations for target mashups. We first execute mashup-tag-service tensor decomposition to obtain the low-dimensional vector representations for mashup, tags, and services, respectively. Next, we present and conduct an interaction-aware heterogeneous neighbor aggregation to aggregate not only the neighboring nodes' embeddings, but also their interaction information. Then, we propose a diffusion-aware homogeneous neighbor aggregation to assign edge weight for each pair of homogeneous nodes, and further aggregate features of neighboring nodes to the central node. Moreover, we utilize a type-aware attention network to update the node embeddings of mashups and services by merging three types of embeddings. Finally, we employ a jointly loss function to train our proposed CTGNN, and top N services with highest predicted scores will be recommended for target mashups. The excellent behavior of CTGNN over the state-of-the-art approaches is proved by carrying out experiments on the real-world ProgrammableWeb dataset. Because node features from both heterogeneous and homogeneous networks are aggregated into the embeddings of mashups and services, our proposed CTGNN achieves remarkable improvements in terms of recommendation accuracy and diversity, and is able to generate satisfactory long-tail service recommendations for target mashups.

In the future, the proposed CTGNN will be further extended to alleviate new service cold-start recommendation. Besides, the time-aware high order propagation mechanism for long-tail service recommendation will also be our next target.

REFERENCES

[1] B. Bai, Y. Fan, W. Tan, and J. Zhang, "DLTSR: A deep learning framework for recommendations of long-tail web services," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 73–85, Jan./Feb. 2020.

[2] H. Yin, B. Cui, J. Li, J. Yao, and C. Chen, "Challenging the long tail recommendation," in *Proc. 38th ACM Int. Conf. Very Large Data Bases*, 2012, pp. 896–907.

[3] B. Cao, X. Liu, M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and Web APIs recommendation for mashup development," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.

[4] C. Wei, Y. Fan, and J. Zhang, "Time-aware service recommendation with social-powered graph hierarchical attention network," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 2229–2240, May/Jun. 2023.

[5] H. Mezni, "Temporal knowledge graph embedding for effective service recommendation," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 3077–3088, Sep./Oct. 2022.

[6] L. Ren and W. Wang, "A granular SVM-based method for top-N web services recommendation," *IEEE Trans. Services Comput.*, vol. 15, no. 1, pp. 457–469, Jan./Feb. 2022.

[7] X. Fan, Y. Hu, Z. Zheng, Y. Wang, P. Brezillon, and W. Chen, "CASR-TSE: Context-aware web services recommendation for modeling weighted temporal-spatial effectiveness," *IEEE Trans. Services Comput.*, vol. 14, no. 1, pp. 58–70, Jan./Feb. 2021.

[8] H. Wang, M. Zhao, X. Xie, W. Li, and M. Guo, "Knowledge graph convolutional networks for recommender systems," in *Proc. 28th ACM World Wide Web Conf.*, 2019, pp. 3307–3313.

[9] Q. Lv et al., "Are we really making much progress?: Revisiting, benchmarking and refining heterogeneous graph neural networks," in *Proc. 27th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 1150–1160.

[10] X. Niu et al., "A dual heterogeneous graph attention network to improve long-tail performance for shop search in E-commerce," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3405–3415.

[11] L. Yao, Q. Sheng, A. Ngy, J. Yu, and A. Segev, "Unified collaborative and content-based web service recommendation," *IEEE Trans. Services Comput.*, vol. 8, no. 3, pp. 453–466, May/Jun. 2015.

[12] B. Chen et al., "TGCN: Tag graph convolutional network for tag-aware recommendation," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 155–164.

[13] B. Chen, Y. Ding, X. Xin, Y. Li, Y. Wang, and D. Wang, "AIRec: Attention intersection model for tag-aware recommendation," *Neurocomputing*, vol. 421, pp. 105–114, Jan. 2021.

[14] R. Huang, N. Wang, C. Han, F. Yu, and L. Cui, "TNAM: A tag-aware neural attention model for top-N recommendation," *Neurocomputing*, vol. 385, pp. 1–12, Apr. 2020.

[15] Y. Pan, Y. Huo, J. Tang, Y. Zeng, and B. Chen, "Exploiting relational tag expansion for dynamic user profile in a tag-aware ranking recommender system," *Inf. Sci.*, vol. 545, pp. 448–464, Feb. 2021.

[16] E. Zheng, G. Kondo, S. Zilora, and Y. Qi, "Tag-aware dynamic music recommendation," *Expert Syst. Appl.*, vol. 106, pp. 244–251, Sep. 2018.

[17] Z. Xu, T. Lukasiewicz, C. Chen, Y. Miao, and X. Meng, "Tag-aware personalized recommendation using a hybrid deep model," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 3196–3202.

[18] M. Shi, J. Liu, D. Zhou, and Y. Tang, "A topic-sensitive method for mashup tag recommendation utilizing multi-relationship service data," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 342–355, Mar./Apr. 2021.

[19] J. Chen, H. Dong, X. Wang, F. Feng, M. Wang, and X. He, "Bias and debias in recommender system: A survey and future directions," *ACM Trans. Inf. Syst.*, vol. 41, no. 3, Jul. 2023, Art. no. 67.

[20] Z. Zhao et al., "Popularity bias is not always evil: Disentangling benign and harmful bias for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 9920–9931, Oct. 2023, doi: 10.1109/TKDE.2022.3218994.

[21] T. Wei, F. Feng, J. Chen, Z. Wu, J. Yi, and X. He, "Model-agnostic counterfactual reasoning for eliminating popularity bias in recommender system," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 1791–1800.

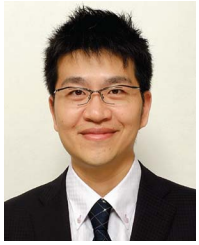
[22] Y. Wang, X. Xin, Y. Ding, and D. Wang, "ICPE: An item cluster-wise pareto-efficient framework for recommendation debiasing," doi: 10.48550/arXiv.2109.12887.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.

[23] Y. Zhang et al., "Causal intervention for leveraging popularity bias in recommendation," in *Proc. 44th ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 11–20.
[24] Z. Chen, R. Xiao, C. Li, G. Ye, H. Sun, and H. Deng, "ESAM: Discriminative domain adaptation with non-displayed items to improve long-tail performance," in *Proc. 43rd ACM SIGIR Conf. Knowl. Discov. Data Mining*, 2020, pp. 579–588.
[25] J. Li, K. Lu, Z. Huang, and H. Shen, "On both cold-start and long-tail recommendation with social data," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 194–208, Jan. 2021.
[26] I. Kanat, T. Raghu, and A. Vinze, "Heads or tails? Network effects on game purchase behavior in the long tail market," *Inf. Syst. Front.*, vol. 22, pp. 803–814, Aug. 2020.
[27] X. Zhou, P. Pan, Y. Zheng, Y. Xu, and R. Jin, "Large scale long-tailed product recognition system at Alibaba," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 3353–3356.
[28] J. Pang, J. Guo, and W. Zhang, "Using multi-objective optimization to solve the long tail problem in recommender system," in *Proc. 23rd Pacific-Asia Conf. Adv. Knowl. Discov. Data Mining*, 2019, pp. 302–313.
[29] Z. Sun, X. Zhang, H. Li, Y. Xiao, and H. Guo, "Recommender systems based on tensor decomposition," *Comput. Mater. Continua*, vol. 66, no. 1, pp. 621–630, Jan. 2021.
[30] Z. Zhang, M. Dong, K. Ota, Y. Zhang, and Y. Ren, "LBCF: A link-based collaborative filtering for over-fitting problem in recommender system," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 6, pp. 1450–1464, Dec. 2021.
[31] Z. Zhang, T. Zhou, and Y. Zhang, "Personalized recommendation via integrated diffusion on user-item-tag tripartite graphs," *Physica A*, vol. 389, pp. 179–186, Jan. 2010.
[32] Z. Zhang, M. Dong, K. Ota, Y. Zhang, and Y. Kudo, "Context-enhanced probabilistic diffusion for urban point-of-interest recommendation," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3156–3169, Nov./Dec. 2022.
[33] Y. Zhong, Y. Fan, K. Huang, W. Tan, and J. Zhang, "Time-aware service recommendation for mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 3, pp. 356–368, May/Jun. 2015.
[34] Z. Zhang, M. Dong, K. Ota, and Y. Kudo, "Alleviating new user cold-start in user-based collaborative filtering via bipartite network," *IEEE Trans. Computat. Social Syst.*, vol. 7, no. 3, pp. 672–685, Jun. 2020.
[35] Z. Zhang, Y. Zhang, and Y. Ren, "Employing neighborhood reduction for alleviating sparsity and cold start problems in user-based collaborative filtering," *Inf. Retrieval J.*, vol. 23, no. 4, pp. 449–472, Jun. 2020.
[36] Z. Zhang, Y. Kudo, T. Murai, and Y. Ren, "Improved covering-based collaborative filtering for new users' personalized recommendations," *Knowl. Inf. Syst.*, vol. 62, no. 5, pp. 3133–3154, Aug. 2020.
[37] Z. Zhang, Y. Kudo, and T. Murai, "Neighbor selection for user-based collaborative filtering using covering-based rough sets," *Ann. Operations Res.*, vol. 256, no. 2, pp. 359–374, Sep. 2017.
[38] B. Sarwar, G. Karypis, J. Knostan, and J. Riedl, "Item-based collaborative filtering recommendation algorithm," in *Proc. 10th Int. Conf. World Wide Web*, 2001, pp. 285–295.
[39] S. Rendle, "Factorization machines with libFM," *ACM Trans. Intell. Syst. Technol.*, vol. 3, no. 3, 2012, Art. no. 57.

Zhipeng Zhang received the PhD degree from the Department of Information and Electronic Engineering, Muroran Institute of Technology, Japan, in 2017. He is currently an associate professor with the School of Computer Science and Artificial Intelligence, Liaoning Normal University, China. He is also a visiting scholar with the Department of Systems Innovation, Osaka University, Japan supported by CSC from 2023 to 2024. He was awarded as the Dalian Young Star in science and technology. His research interests include recommender systems, data mining, and artificial intelligence.

Yuhang Zhang received the BS degree in computer technology from Liaoning Normal University, Dalian, China, in 2021. He is currently working toward the MS degree in computer technology with the Liaoning Normal University, Dalian, China. His research interests include recommender systems and data mining.



Mianxiong Dong (Member, IEEE) received the BS, MS, and PhD degrees in computer science and engineering from the University of Aizu, Japan. He is vice president and the youngest ever professor with the Muroran Institute of Technology, Japan. He was a JSPS research fellow with theSchool of Computer Science and Engineering, University of Aizu, Japan and was a visiting scholar with BBCR Group, University of Waterloo, Canada supported by JSPS Excellent Young Researcher Overseas Visit Program from 2010 to 2011. He was selected as a foreigner research fellow (a total of 3 recipients all over Japan) by NEC C&C Foundation in 2011. He is the recipient of IEEE TCSC Early Career Award 2016, IEEE SCSTC Outstanding Young Researcher Award 2017, the 12th IEEE ComSoc Asia-Pacific Young Researcher Award 2017, Funai Research Award 2018 and NISTEP Researcher 2018 (one of only 11 people in Japan) in recognition of significant contributions in science and technology. He is Clarivate Analytics 2019 Highly cited researcher (Web of Science) and foreign fellow of the EAJ.

Kaoru Ota (Member, IEEE) received the BS degree in computer science and engineering from the University of Aizu, Japan, in 2006, the MS degree in computer science from Oklahoma State University, USA, in 2008, and the PhD degree in computer science and engineering from the University of Aizu, Japan, in 2012. She is a professor and ministry of education, culture, sports, science and technology (MEXT) Excellent Young researcher with the Department of Sciences and Informatics, Muroran Institute of Technology, Japan. From March 2010 to March 2011, she was a visiting scholar with the University of Waterloo, Canada. Also, she was a Japan Society of the Promotion of Science (JSPS) research fellow with Tohoku University, Japan from April 2012 to April 2013. She is the recipient of IEEE TCSC Early Career Award 2017, the 13th IEEE ComSoc Asia-Pacific Young Researcher Award 2018, 2020 N2Women: Rising Stars in Computer Networking and Communications, 2020 KDDI Foundation Encouragement Award, and 2021 IEEE Sapporo Young Professionals Best Researcher Award. She is Clarivate Analytics 2019, 2021 highly cited researcher (Web of Science) and is selected as JST-PRESTO researcher in 2021, fellow of the EAJ in 2022.

Yao Zhang received the PhD degree from the Muroran Institute of Technology, Japan, in 2017. She is currently a lecture with the School of Mechanical Engineering and Automation, Dalian Polytechnic University, China. Her research interests include material performance prediction and data analysis.

Yonggong Ren received the PhD degree from the College of Computer Science and Engineering, Northeastern University, China, in 2006. He is currently a professor and dean with the School of Computer Science and Artificial Intelligence, Liaoning Normal University, China. His research interests include database, data mining, and artificial intelligence.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:08:07 UTC from IEEE Xplore. Restrictions apply.